

Dynamic Locomotion and Whole-Body Control for Quadrupedal Robots

C. Dario Bellicoso, Fabian Jenelten, Péter Fankhauser, Christian Gehring, Jemin Hwangbo, Marco Hutter

Abstract—This paper presents a framework which allows a quadrupedal robot to execute dynamic gaits including trot, pace and dynamic lateral walk, as well as a smooth transition between them. Our method relies on an online ZMP based motion planner which continuously updates the reference motion trajectory as a function of the contact schedule and the state of the robot. The planner is coupled with a hierarchical whole-body controller which optimizes the whole-body motion and contact forces by solving a cascade of prioritized tasks. We tested our framework on ANYmal, a fully torque controllable quadrupedal robot which is actuated by series-elastic actuators.

I. INTRODUCTION

Research focused on robotic locomotion has made great efforts in trying to reach the natural capabilities of animals, which are capable of executing highly dynamic gaits and naturally perform quick and smooth transitions from one gait to another. The recent advances in both the motion planning and control methods, as well as the increase in performance of both hardware and software, are narrowing the gap between legged robots and their biological counterparts. Although we are still far from a fully autonomous system capable of locomotion in all environments, the recent years have seen several important results.

Dynamic locomotion is a complex problem which involves a multi-body system which interacts with the environment through several contact points. The execution of dynamic gaits for such a system over unknown and challenging terrain requires careful motion planning and accurate motion and force control. Research groups and companies have demonstrated the ability to walk, run and jump over challenging terrain. *Cheetah 2* [15] from MIT has been shown able to run and jump over obstacles which are 80% of the robot's leg length using an MPC controller. Recent results have shown *ANYmal* [9] walking [1], trotting [5], and climbing over stairs [3] by employing whole-body control and motion optimization algorithms. The IIT *HyQ* [17] quadruped has been shown walking using a creeping gait [14] over stepping stones [19]. Motion planning for the torso was obtained by solving an optimization problem which constrained the *Zero-Moment Point* (ZMP) [18] to lie in the support polygons.

A notable example of dynamic locomotion is given by the legged robots developed by Boston Dynamics. One of their recent legged robots, *Spot*, has demonstrated a wide variety of dynamic gaits, including a dynamic walk, trotting with

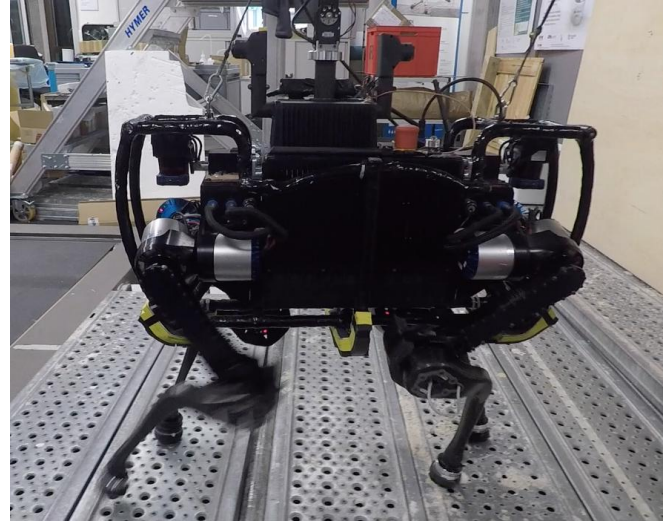


Fig. 1. Our motion plan and control framework, tested on the fully torque-controllable quadrupedal robot ANYmal, can execute a dynamic walk which includes overlapping swing phases for lateral legs (e.g. left-front and left-hind legs).

full flight phases, pacing, as well as demonstrating smooth transitions between these. *Spot mini*, a more recent machine, has shown similar skills as well as manipulation capability with a robotic arm. While impressive, the methods to achieve such results are not available. In this paper, we contribute a novel framework and methods for motion planning and control algorithms needed to execute complex and dynamic gaits.

Based on the results shown in [10] and [19], we developed a ZMP based motion planner which can generate a motion through a sequence of arbitrary support polygons, including triangles and quadrilaterals, but also lines, which is the case for a dynamic walk with overlapping swing phases or for a trotting and pacing gait. Rather than computing the optimal motion on discrete events (e.g. a foot touchdown), we recompute the optimization as soon as the previous one ends successfully, providing us with a motion plan which continuously adapts to the new state of the robot as well as to changes in the gait pattern. We also reformulate the cost function to include costs on the deviation from previous solutions, as well as costs which penalize the deviation from a path regularizer. Motion tracking is obtained by means of a whole-body controller which solves a sequence of motion and contact force prioritized tasks using the hierarchical optimization formulation. We demonstrate our method by executing dynamic gaits such as a dynamic lateral walk with overlapping swing phases, a trot and a pace on the

This work was supported in part by the Swiss National Science Foundation (SNF) through the National Centre of Competence in Research Robotics.

All authors are with the Robotic Systems Lab, ETH Zurich, Switzerland, bellicoso@mavt.ethz.ch

quadrupedal robot ANYmal. We conclude our experiments by showing a transition between a dynamic lateral walk and a pace gait. To the best of our knowledge, this is the first contribution with results and methodologies on the execution of such dynamic gaits, specifically a dynamic lateral walk, a pacing gait and transitions between them.

II. MODEL FORMULATION

In general, a walking robot can be modeled as a system with a free-floating base B to which legs are attached (see Fig.2). The motion of the entire system can be described w.r.t. a fixed inertial frame I . We write the position vector from the inertial to the base frame expressed in the inertial frame as ${}^I\mathbf{r}_{IB} \in \mathbb{R}^3$ and use Hamiltonian unit quaternions to parametrize the orientation of the main body. The limb joint angles are stacked in the vector $\mathbf{q}_j \in \mathbb{R}^{n_j}$. We write the generalized coordinates vector $\mathbf{q}$ and the generalized velocities vector $\mathbf{u}$ as

$$\mathbf{q} = \begin{bmatrix} {}^I\mathbf{r}_{IB} \\ \mathbf{q}_{IB} \\ \mathbf{q}_j \end{bmatrix} \in SE(3) \times \mathbb{R}^{n_j}, \quad \mathbf{u} = \begin{bmatrix} {}^I\mathbf{v}_B \\ {}^B\boldsymbol{\omega}_{IB} \\ \dot{\mathbf{q}}_j \end{bmatrix} \in \mathbb{R}^{n_u}, \quad (1)$$

where $n_u = 6 + n_j$, ${}^I\mathbf{v}_B$ and ${}^B\boldsymbol{\omega}_{IB}$ are the linear and angular velocity of the base w.r.t. the inertial frame expressed respectively in the I and B frame, and $\mathbf{q}_{IB} \in SO(3)$ is the unit quaternion that projects the components of a vector expressed in B frame to those of the same vector expressed in I frame. The equations of motion of legged systems can be written as $\mathbf{M}(\mathbf{q})\dot{\mathbf{u}} + \mathbf{h}(\mathbf{q}, \mathbf{u}) = \mathbf{S}^T\boldsymbol{\tau} + \mathbf{J}_s^T\boldsymbol{\lambda}$, where $\mathbf{M}(\mathbf{q}) \in \mathbb{R}^{n_u \times n_u}$ is the mass matrix and $\mathbf{h}(\mathbf{q}, \mathbf{u}) \in \mathbb{R}^{n_u}$ is the vector of Coriolis, centrifugal and gravity terms. The selection matrix $\mathbf{S} = [\mathbf{0}_{n_\tau \times (n_u - n_\tau)} \quad \mathbf{I}_{n_\tau \times n_\tau}]$ selects which DoFs are actuated. If all limb joints are actuated, then $n_\tau = n_j$. The vector of constraint forces $\boldsymbol{\lambda}$ is mapped to the joint space torques through the support Jacobian $\mathbf{J}_s \in \mathbb{R}^{3n_c \times n_u}$, which is obtained by stacking the constraint Jacobians as $\mathbf{J}_s = [\mathbf{J}_{C_1}^T \cdots \mathbf{J}_{C_{n_c}}^T]^T$, with n_c the number of limbs in contact. Motion at the supporting contact points must be constrained. If the feet are modeled as point contacts, then each contact introduces three constraint equations ${}^I\mathbf{r}_{IC}(t) = \text{const}$, which can be differentiated twice to yield

$${}^I\dot{\mathbf{r}}_{IC} = \mathbf{J}_C\mathbf{u} = \mathbf{0}, \quad {}^I\ddot{\mathbf{r}}_{IC} = \mathbf{J}_C\dot{\mathbf{u}} + \dot{\mathbf{J}}_C\mathbf{u} = \mathbf{0}. \quad (2)$$

III. HIERARCHICAL OPTIMIZATION

We have recently shown [1] the implementation of a whole body controller which finds the optimal joint torques and joint accelerations by solving a cascade of prioritized QP problems. In this work we use a different implementation which solves for contact forces rather than joint torques. Hence, we search for a vector $\boldsymbol{\xi}_d = [\dot{\mathbf{u}}_d^T \quad \boldsymbol{\lambda}_d^T]^T \in \mathbb{R}^{n_u + n_c}$ of desired joint accelerations and contact forces. The dimension of $\boldsymbol{\xi}_d$ is a function of the number of contact points n_c . When the feet are modeled as point contacts, the dimensions of $\boldsymbol{\xi}_d$ will be smaller or equal to the case of optimizing for joint torques. As noted in [8], this is beneficial in terms of computation speed when solving the numerical optimization

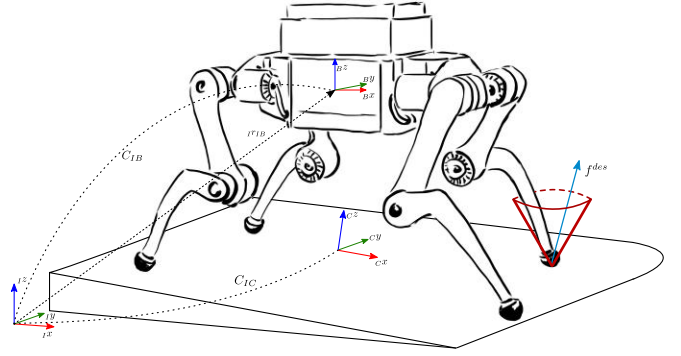


Fig. 2. A sketch of the quadrupedal robot ANYmal. The floating base pose is defined by the position of the base frame w.r.t. the inertial frame expressed in the inertial frame ${}^I\mathbf{r}_{IB}$, and by the orientation of the base w.r.t. the inertial frame $\mathbf{C}_{IB}$. As discussed in earlier works [5], we define a control frame $\mathbf{C}_{IC}$ which is aligned with the local estimation of the terrain and with the robot's heading direction.

of tasks. It also results in an more natural way of expressing constraints of contact forces.

Equations of motion: By taking advantage of the decomposition induced by the selection matrix $\mathbf{S}^T$, we can constrain the motion and the contact forces to live on the manifold described by the floating base system dynamics by writing

$$[\mathbf{M}_{fb} \quad -\mathbf{J}_{sfb}^T] \boldsymbol{\xi}_d = -\mathbf{h}_{fb}, \quad (3)$$

where $\mathbf{M}_{fb}$, $\mathbf{J}_{sfb}^T$ and $\mathbf{h}_{fb}$ are the first six rows of the composite inertia matrix, the support Jacobian and the nonlinear terms respectively, which are the equations related to the dynamics of the floating base.

Contact motion constraint: The solution found by the controller should not violate the contact constraints defined in (2). Hence, we impose null accelerations at the contact points by setting

$$[\mathbf{J}_s \quad \mathbf{0}_{3n_c \times 3n_c}] \boldsymbol{\xi}_d = -\dot{\mathbf{J}}_s\mathbf{u}. \quad (4)$$

Contact force and torque limits: To avoid slipping, contact forces should be constrained to lie in the friction cone which is aligned with the normal to the contact surface ${}^I\mathbf{n}$ expressed in world frame and is a function of the friction coefficient μ . To write these constraints as linear ones, we approximate the friction cones with pyramids. These are aligned with the unit perpendicular vectors ${}^I\mathbf{l}$ and ${}^I\mathbf{h}$ which lie on the contact surface. We achieve this by writing constraints on the k -th contact force $\boldsymbol{\lambda}_k$ as

$$\begin{aligned} ({}^I\mathbf{h} - {}^I\mathbf{n}\mu)^T {}^I\boldsymbol{\lambda}_k &\leq 0 \\ -({}^I\mathbf{h} + {}^I\mathbf{n}\mu)^T {}^I\boldsymbol{\lambda}_k &\leq 0 \\ ({}^I\mathbf{l} - {}^I\mathbf{n}\mu)^T {}^I\boldsymbol{\lambda}_k &\leq 0 \\ -({}^I\mathbf{l} + {}^I\mathbf{n}\mu)^T {}^I\boldsymbol{\lambda}_k &\leq 0. \end{aligned} \quad (5)$$

Joint actuation torques should also be limited to avoid mechanical limits given by the actuators. We can write

$$\boldsymbol{\tau}_{min} - \mathbf{h}_j \leq [\mathbf{M}_j \quad -\mathbf{J}_{sj}^T] \boldsymbol{\xi}_d \leq \boldsymbol{\tau}_{max} - \mathbf{h}_j, \quad (6)$$

TABLE I

THE LIST OF PRIORITIZED TASKS USED IN OUR EXPERIMENTS. EACH TASK IS ASSOCIATED WITH A PRIORITY (1 IS THE HIGHEST).

Priority	Task
1	Floating base equations of motion
2	Torque limits Friction cone and λ modulation
3	No contact motion
4	Center of mass linear motion Center of mass angular motion
5	Swing leg motion tracking Contact force minimization

where the j subscript is relative to the rows of the equations of motion describing the joint dynamics. The quantities τ_{min} and τ_{max} are n_j dimensional vectors of minimum and maximum actuator torques.

Motion tracking: To track the desired motion of the floating base and the swing legs, we constrain the joint accelerations by implementing operational space controllers with feed-forward reference acceleration and a motion dependent state feedback state. For the main body we write

$$\begin{bmatrix} C\mathbf{J}_P & \mathbf{0} \end{bmatrix} \xi_d = C\ddot{\mathbf{r}}_{IB}^d + \mathbf{k}_D^{pos}(C\dot{\mathbf{r}}_{IB}^d - C\mathbf{v}_B) + \mathbf{k}_P^{pos}(C\mathbf{r}_{IB}^d - C\mathbf{r}_{IB}) \quad (7)$$

for the linear motion and

$$\begin{bmatrix} C\mathbf{J}_R & \mathbf{0} \end{bmatrix} \xi_d = -\mathbf{k}_D^{ang} C\omega_B + \mathbf{k}_P^{ang}(\mathbf{q}_{CB}^d \boxminus \mathbf{q}_{CB}) \quad (8)$$

for the angular motion. The Jacobian matrices $C\mathbf{J}_P$ and $C\mathbf{J}_R$ are the translational and rotational Jacobian associated to the base expressed in the *Control frame* C , which is a frame aligned with the local estimation of the terrain and with the heading direction of the robot (for more details see [1] and [5]). The $\boxminus$ operator yields the Euler vector which represents the relative orientation between the desired attitude $\mathbf{q}_{CB}^d$ and the estimated one $\mathbf{q}_{CB}$. The gains $\mathbf{k}_P^{pos}$, $\mathbf{k}_D^{pos}$, $\mathbf{k}_P^{ang}$ and $\mathbf{k}_D^{ang}$ are diagonal positive definite matrices of control gains. The reference motion $C\mathbf{r}_{IB}$ and its derivatives are the output of the motion plan which is described in section IV.

Contact force minimization: We minimize the contact forces by setting

$$\begin{bmatrix} \mathbf{0}_{3n_c \times n_u} & \mathbb{I}_{3n_c \times 3n_c} \end{bmatrix} \xi_d = \mathbf{0}. \quad (9)$$

Table I summarizes the tasks and their priority (a lower value indicates a higher priority) used throughout the experiments.

Given a computed set of joint motions and contact forces $\xi_d = [\dot{\mathbf{u}}_d^T \ \lambda_d^T]^T$, we compute the reference joint torques as $\tau_d = [\mathbf{M}_j \ -\mathbf{J}_{s_j}^T] \xi_d + \mathbf{h}_j$, where $\mathbf{M}_j$, $\mathbf{J}_{s_j}^T$ and $\mathbf{h}_j$ are defined as in (6).

IV. MOTION OPTIMIZATION

We describe how to obtain a motion plan for the x and y coordinates of the whole-body center of mass (com) such that balance is ensured during locomotion at all times. Fig.3 shows the resulting motion plan obtained during the execution of a dynamic lateral walk.

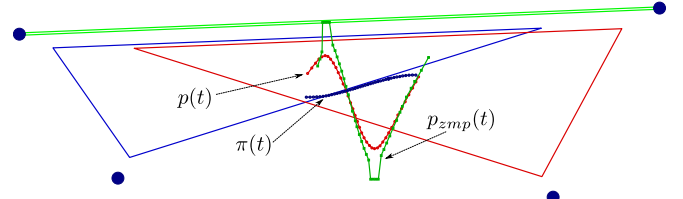


Fig. 3. The solution obtained by the motion planner when a dynamic lateral walk gait is active. The support polygons are computed as the convex hull of the points which are in contact (large blue dots) according to the contact schedule. To increase robustness of the solution, a safety margin is applied to the polygons such that their area is reduced. Only the first three support polygons of the sequence (respectively in red, green and blue) are shown. The resulting center of mass motion $\mathbf{p}(t)$ (red path) is optimized to deviate the least possible from a path regularizer $\pi(t)$ (blue path), while the ZMP $\mathbf{p}_{zmp}(t)$ (green path) is constrained to always lie in the active support polygon. When two lateral legs are simultaneously swinging, the support polygon collapses to a line (shown in green). To avoid numerical issues and excessive accelerations due to a required increase in the order of the approximating spline, we expand the line to a rectangle.

The locomotion framework can receive high-level velocity commands from an external source, i.e. an operator device or a high-level navigation planner. To drive locomotion to the reference speed, footholds are generated for all legs (section IV-A) to obtain the desired average velocity of the torso. This information, together with the footfall pattern (e.g. fig.4), is used to generate a sequence of support polygons (section IV-B) which are sent to the motion plan optimizer (section IV-C). This in turn will produce position, velocity and acceleration profiles for the x and y coordinates of the whole-body center of mass.

The entire plan is computed in the *Plan frame* P which is located at the origin of the inertial frame and follows the yaw rotation of the torso of the robot. Hence, it is always aligned to the heading direction of the robot. This allows us to interpret the problem formulation contributions to the x and y coordinates of the plan as constraints on heading and lateral directions of the motion.

A. Foothold generation

External high-level velocity commands are used to drive locomotion in a specific direction and speed by adjusting the reference footholds, which are computed for each leg at each new control loop. Depending on the contact state of a leg, these are computed in two different ways: when a leg is in contact, the commanded velocities will be projected to foothold locations computed such that the average torso speed matches the required locomotion speed; if a leg is swinging, the reference foothold is computed in the same way but added to a velocity feedback term which stabilizes the robot when it receives a push which produces a velocity control error. For more details, see [5].

B. Support polygon sequence

We describe a gait by defining lift-off ϕ_{lo} and touch-down ϕ_{td} events in the phase domain for each leg. This also defines a contact schedule for all legs. Fig.4 shows the phase events for a dynamic lateral walk, which exhibits overlapping swing

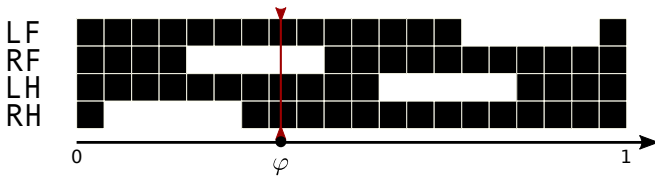


Fig. 4. The contact schedule associated to a dynamic lateral walk. The dark areas represent a contact phase, while the light areas represent a swing phase. The abbreviations stand for left fore (LF), right fore (RF), left hind (LH) and right hind (RH). This gait exhibits an overlap in the swing phases of the left fore/hind and the right fore/hind legs. The red line represents the current phase $\varphi \in [0, 1]$

phases for fore/hind left and fore/hind right legs. Given the touch-down and lift-off events for all legs, as well as the current feet locations and the set of desired footholds, we can compute a sequence of support polygons (defined as the tuple of vertices and time duration in seconds) to be used in the motion planner. We do this whenever a new motion plan is available, such that the new solution adapts to changes in the contact schedule, changes in the reference footholds and changes in the high-level operator velocity. To compute each polygon, we start from the current phase φ and store the $x - y$ coordinates of the feet which are in contact. We search for the next phase event φ_k with $k = 1$ to get the duration of the first polygon $t_0 = t_s(\varphi_k - \varphi)$, with t_s the stride duration in seconds. Thus, we have completely defined a support polygon by its vertices and its duration in seconds. We iterate by updating φ to φ_k and searching for the next phase event. Since the contact schedule is periodic, we repeat these steps until the phase event $\varphi_0 + 1$, which corresponds to the starting phase φ_0 .

C. Problem formulation

As done in [19], we choose to represent the motion plan by using two sequences of fifth order polynomial splines for the x and y coordinates of the center of mass position. Each spline has a time duration t_{fk} . Hence, the position $\mathbf{p} \in \mathbb{R}^2$ at time t described by the k -th pair of splines in the sequence can be compactly written as

$$\mathbf{p}(t) = \begin{bmatrix} p_x(t) \\ p_y(t) \end{bmatrix} = \begin{bmatrix} \alpha_{kx}^T \\ \alpha_{ky}^T \end{bmatrix} \boldsymbol{\eta}(t) \quad (10)$$

where

$$\begin{aligned} \alpha_{kx}^T &= [a_{k5}^x \ a_{k4}^x \ a_{k3}^x \ a_{k2}^x \ a_{k1}^x \ a_{k0}^x] \\ \alpha_{ky}^T &= [a_{k5}^y \ a_{k4}^y \ a_{k3}^y \ a_{k2}^y \ a_{k1}^y \ a_{k0}^y] \end{aligned} \quad (11)$$

and

$$\boldsymbol{\eta}(t)^T = [t^5 \ t^4 \ t^3 \ t^2 \ t \ 1]. \quad (12)$$

Alternatively we can write

$$\mathbf{p}(t) = \begin{bmatrix} \boldsymbol{\eta}(t)^T & \mathbf{0}_{6 \times 1} \\ \mathbf{0}_{6 \times 1} & \boldsymbol{\eta}(t)^T \end{bmatrix} \begin{bmatrix} \alpha_{kx}^T \\ \alpha_{ky}^T \end{bmatrix} = \mathbf{T}(t) \boldsymbol{\alpha}_k, \quad (13)$$

with $\mathbf{T}(t) \in \mathbb{R}^{2 \times 12}$. We hence represent the k -th pair of splines as $\mathbf{s}_k^T = [\alpha_{kx}^T \ \alpha_{ky}^T]$. Using this parametrization, we setup an optimization problem that will solve for all the

spline coefficients for $k = 0, \dots, n - 1$, with n the number of splines used to generate the motion. This also allows to easily represent velocities and accelerations as

$$\dot{\mathbf{p}}(t) = \begin{bmatrix} \alpha_{kx}^T \\ \alpha_{ky}^T \end{bmatrix} \dot{\boldsymbol{\eta}}(t) \quad \ddot{\mathbf{p}}(t) = \begin{bmatrix} \alpha_{kx}^T \\ \alpha_{ky}^T \end{bmatrix} \ddot{\boldsymbol{\eta}}(t) \quad (14)$$

or by writing $\dot{\mathbf{p}}(t) = \dot{\mathbf{T}}(t) \boldsymbol{\alpha}_k$ and $\ddot{\mathbf{p}}(t) = \ddot{\mathbf{T}}(t) \boldsymbol{\alpha}_k$. The problem we solve is then to search for the vector optimal coefficients

$$\boldsymbol{\xi} = \begin{bmatrix} \alpha_{0x}^T & \alpha_{0y}^T & \alpha_{1x}^T & \alpha_{1y}^T & \dots & \alpha_{(n-1)x}^T & \alpha_{(n-1)y}^T \end{bmatrix}^T \quad (15)$$

by solving a QP problem. The total number of splines used to approximate the motion is $2n$. The following will describe the setup of an optimization problem formulated as

$$\begin{aligned} \min_{\boldsymbol{\xi}} \quad & \frac{1}{2} \boldsymbol{\xi}^T \mathbf{Q} \boldsymbol{\xi} + \mathbf{c}^T \boldsymbol{\xi} \\ \text{s. t.} \quad & \mathbf{A} \boldsymbol{\xi} = \mathbf{b}, \quad \mathbf{D} \boldsymbol{\xi} \leq \mathbf{f}. \end{aligned} \quad (16)$$

In the following, if not explicitly noted otherwise, we will refer only to the x coordinate of each pair of splines $\mathbf{s}_k$. The same arguments hold for the y coordinate.

D. Plan initialization

The motion plan is initialized with an extended state (position, velocity and acceleration) which is used as a hard constraint for the initial state of the spline sequence. This is done by fusing together the previous desired position $\mathbf{c} \mathbf{r}_{IB}^{des}$ with the current measured one $\mathbf{c} \mathbf{r}_{IB}$ by using an exponentially decaying alpha filter designed as

$$\mathbf{c} \mathbf{r}_{IB}^{ref} = \alpha \mathbf{c} \mathbf{r}_{IB}^{des} + (1 - \alpha) \mathbf{c} \mathbf{r}_{IB} \quad (17)$$

with $\alpha = 0.5e^{-\lambda t_c}$, where $\lambda \in \mathbb{R}$ can be set to weigh the desired position as a function of the computation time t_c since the last successful optimization. A similar filter is employed to set the initial velocities, whereas acceleration is initialized with the last available reference one.

E. Cost function

In our formulation, several terms contribute to the Hessian matrix $\mathbf{Q} \in \mathbb{R}^{12n \times 12n}$ and the linear term $\mathbf{c} \in \mathbb{R}^{12n}$ appearing in (16). We penalize the deviation of the position $\mathbf{p}(t) = \mathbf{T}(t) \boldsymbol{\alpha}$ from a desired position reference $\mathbf{p}_r$ by writing

$$\begin{aligned} & \frac{1}{2} \|\mathbf{T} \boldsymbol{\alpha} - \mathbf{p}_r\|_{\mathbf{W}}^2 \\ &= \frac{1}{2} (\mathbf{T} \boldsymbol{\alpha} - \mathbf{p}_r)^T \mathbf{W} (\mathbf{T} \boldsymbol{\alpha} - \mathbf{p}_r) \\ &= \frac{1}{2} \boldsymbol{\alpha}^T \mathbf{T}^T \mathbf{W} \mathbf{T} \boldsymbol{\alpha} - \mathbf{p}_r^T \mathbf{W} \mathbf{T} \boldsymbol{\alpha} + \frac{1}{2} \mathbf{p}_r^T \mathbf{W} \mathbf{p}_r. \end{aligned} \quad (18)$$

Minimizing the squared norm in (18) is equivalent to solving a QP in the form 16 with

$$\mathbf{Q} = \mathbf{T}^T \mathbf{W} \mathbf{T} \quad \mathbf{c} = -\mathbf{T}^T \mathbf{W} \mathbf{p}_r. \quad (19)$$

To penalize deviation for velocity references, it is simply required to use $\dot{\mathbf{T}}$ and $\dot{\mathbf{p}}_r$ in (18). A similar reasoning can be done for acceleration deviations.

1) *Acceleration minimization*: As shown in [10], we can minimize the acceleration by writing

$$\mathbf{Q}_k^{acc} = \begin{bmatrix} (400/7)t_f^7 & 40t_f^6 & 24t_f^5 & 10t_f^4 & & \\ 40t_f^6 & 28.8t_f^5 & 18t_f^4 & 8t_f^3 & \vdots & \\ 24t_f^5 & 18t_f^4 & 12t_f^3 & 6t_f^2 & \mathbf{0}_{4 \times 2} & \\ 10t_f^4 & 8t_f^3 & 6t_f^2 & 4t_f & \vdots & \\ \cdots & \mathbf{0}_{2 \times 4} & \cdots & & \mathbf{0}_{2 \times 2} & \end{bmatrix} \quad (20)$$

with a null linear term $\mathbf{c}_k^{acc} = \mathbf{0}$. Note that if this were the only term added to the cost function, there would be no cost associated to the α_{1k} and α_{2k} coefficients of each spline, leading to a positive semi-definite Hessian. This is problematic when using a method such as the Active Set one [6] which requires the Hessian to be positive definite. In that case, a regularizer term can be added as

$$\mathbf{Q}_k^{acc\rho} = \begin{bmatrix} \mathbf{0}_{4 \times 4} & \mathbf{0}_{4 \times 2} \\ \mathbf{0}_{2 \times 4} & \rho \mathbb{I}_{2 \times 2} \end{bmatrix} \quad (21)$$

with $\rho = 10e^{-8}$ and a null linear term.

2) *Soft final constraints*: We set the final position $\mathbf{p}_f \in \mathbb{R}^2$ to be at the center $\mathbf{p}_f^{ref} \in \mathbb{R}^2$ of the polygon defined by the planned footholds, which is the polygon that would support the robot if it would stop to stand at the end of the support polygon sequence. To minimize the norm $\|\mathbf{p}_f - \mathbf{p}_f^{ref}\|_{\mathbf{W}_f}^2 = \|\mathbf{A}_f \mathbf{s}_f - \mathbf{p}_f^{ref}\|_{\mathbf{W}_f}^2$ we write

$$\mathbf{Q}_f = \mathbf{A}_f^T \mathbf{W}_f \mathbf{A}_f \quad \mathbf{c}_f = -\mathbf{A}_f^T \mathbf{W}_f \mathbf{p}_f^{ref}, \quad (22)$$

with $\mathbf{W}_f \in \mathbb{R}^{2 \times 2}$ a symmetric diagonal matrix of positive weights. To avoid solutions in which the optimizer places the final state far away from the reference position, we add inequality constraints on the position such that it is constrained in a box centered at the reference position.

3) *Deviation from previous solution*: Since we are computing a new optimization as soon as the previous one was successful, to avoid large deviations between successive motion plans we penalize deviations on position, velocity and acceleration obtained by the current solution ξ from the ones resulting from the previous one ξ_{i-1} . Given t_{pre} the time elapsed since the last successful optimization, we penalize the deviations $\|\mathbf{p}(\bar{t}) - \mathbf{p}_{i-1}(t_{pre} + \bar{t})\|_{\mathbf{W}_{pre}}^2$, $\forall \bar{t} \in [0, t_f]$, where $t_f = \sum_{k=0}^{n-1} t_{fk}$ is the sum of the durations of all of the n splines that compose the motion. We discretize the optimization horizon $[0, t_f]$ using a sample time dt . We employ a similar cost penalizing deviations from velocities and accelerations obtained by the last solution.

4) *Path regularization*: Continuous update of the motion plan can cause drift in the motion of the torso w.r.t. the reference footholds. This can be caused by the accumulation of control errors, which alter the solution such that the motion can become unfeasible. To avoid this issue, we add a cost on the deviation from a reference *regularizer path*. The path itself is approximated as a sequence of splines represented by $\pi(t)$, $\dot{\pi}(t)$ and $\ddot{\pi}(t)$. The spline coefficients of the path regularizer are obtained from a minimization problem setup such that:

- The initial state $\pi(0)$, $\dot{\pi}(0)$ and $\ddot{\pi}(0)$ coincides with the center of the initial support polygon and is equal to the initial velocity and acceleration
- The final state $\pi(t_f)$, $\dot{\pi}(t_f)$ and $\ddot{\pi}(t_f)$ is set as in section IV-E.2
- Acceleration is minimized
- Splines are smoothly connected by setting equality constraints on the state at the spline junctions

By sampling the entire motion as done in section IV-E.3, we penalize deviations of the resulting motion from the path regularizer.

F. Equality constraints

Since the entire motion is composed of a sequence of splines, we setup the problem to ensure that they are connected. Since the initial state cannot be modified by the motion plan, we set a hard equality constraint such that the initial state of the first spline $\mathbf{s}_0$ coincides with the one set in the plan initialization. The initial hard constraint can be written as $\mathbf{p}(0) = \mathbf{p}^r$, $\dot{\mathbf{p}}(0) = \dot{\mathbf{p}}^r$ and $\ddot{\mathbf{p}}(0) = \ddot{\mathbf{p}}^r$. Hence, we write

$$\begin{bmatrix} \eta(0)^T \\ \dot{\eta}(0)^T \\ \ddot{\eta}(0)^T \end{bmatrix} \alpha_0^x = \begin{bmatrix} \mathbf{p}_x^r \\ \dot{\mathbf{p}}_x^r \\ \ddot{\mathbf{p}}_x^r \end{bmatrix}. \quad (23)$$

To ensure that two splines $\mathbf{s}_k$ and $\mathbf{s}_{k+1}$ are connected, we constrain them such that

$$\begin{bmatrix} \eta(t_{fk})^T & -\eta(0)^T \\ \dot{\eta}(t_{fk})^T & -\dot{\eta}(0)^T \end{bmatrix} \begin{bmatrix} \alpha_k^x \\ \alpha_{k+1}^x \end{bmatrix} = \mathbf{0}, \quad (24)$$

with t_{fk} representing the duration in seconds of spline $\mathbf{s}_k$. When the equality junction connecting two splines lies between two disjoint support polygons, we cannot guarantee the smoothness or the motion anymore, but will have to allow the ZMP to jump, which will cause a jump in the acceleration reference. Although this has a negative effect on the controller, it does allow the optimizer to find a solution when traversing disjoint support polygons. We check whether two polygons intersect by using the *Separating Axis Theorem* (SAT) described in [7]. If such an intersection exists, we also constrain acceleration between two splines by writing

$$\begin{bmatrix} \ddot{\eta}(t_f)^T & -\ddot{\eta}(0)^T \end{bmatrix} \begin{bmatrix} \alpha_k^x \\ \alpha_{k+1}^x \end{bmatrix} = \mathbf{0}. \quad (25)$$

G. Inequality constraints

As demonstrated in [11] and [19], balance during locomotion can be ensured by constraining the *Zero-Moment Point* (ZMP) to lie inside the support polygon. Starting from the measured feet configuration at the time the motion plan is called and using the planned feet positions, we compute a sequence of support polygons and their duration by using the contact schedule as defined in [5]. By assuming at least two feet on the ground at all times, the support polygon for a quadrupedal robot can be a line, a triangle or a quadrilateral.

As discussed in [16] and [19], the location of the ZMP is a function of the motion of the center of mass. The location of the x coordinate of the ZMP is defined by

$$x_{zmp} = x_{com} - \frac{z_{com} \ddot{x}_{com}}{g + \ddot{z}_{com}}, \quad (26)$$

where g is the gravitational acceleration. We reorder the vertices of each support polygon counter-clockwise by computing their polar coordinates w.r.t. the center of the support polygon and comparing their phase. We can then constrain the ZMP by adding one constraint for each edge of the support polygon by writing $ax_{zmp} + by_{zmp} + c \geq 0$, where a , b and c are the coefficients of the line that passes through the vertices of an edge of the support polygon. The normal vector to this line $\mathbf{n} = [a \ b]^T$ is directed towards the inside of the polygon.

As mentioned in section IV-E, we place additional inequality constraints on the final position $\mathbf{p}_f$ of the motion. This is obtained by adding four constraints of the form (26) on $\mathbf{p}_f$.

H. Constraint relaxation

The optimizer can fail if the constraints are too tight. First, the initial ZMP may not lie in the current support polygon. For this reason, we exclude the ZMP constraint on the first sample of the motion plan. This way the optimizer is still able to find a solution, and our experiments have shown this to work on the real system.

Second, the support polygon safety margin may be too high. To counteract this, whenever an optimization fails, we iteratively decrease the margin by a fixed amount, and we increase it back at a slower rate to ensure the success of the optimization.

Finally, we check for the time duration t_{fk} associated to each polygon. If it is smaller or comparable to the sample time used to discretize the motion and to setup the constraints, we remove the support polygon from the sequence.

V. EXPERIMENTS

A. Setup

Our experiments¹ were conducted on ANYmal [9], an accurately torque-controllable quadrupedal robot. Control signals are generated in a 400Hz control loop which runs on a dedicated on-board computer together with state estimation [2] and communication with the drives. For modeling and computation of kinematics and dynamics, we use the open-source Rigid Body Dynamics Library [13] (RBDL), which is a C++ implementation of the algorithms described in [4]. To numerically solve the QP problems described in the previous sections, we use a custom version of the QuadProg++ [12] library, a C++ open-source QP solver which implements the Active Set algorithm described in [6].

B. Dynamic locomotion

We have tested our planning and control framework by executing several gaits, including a dynamic lateral walk with overlapping swing phases; a trotting gait; a pacing gait; a transition between dynamic lateral walk and pacing (see Fig.7). The execution of all these gaits was successful both on flat terrain and on slightly uneven terrain, including locomotion over a small but unperceived step. We have also

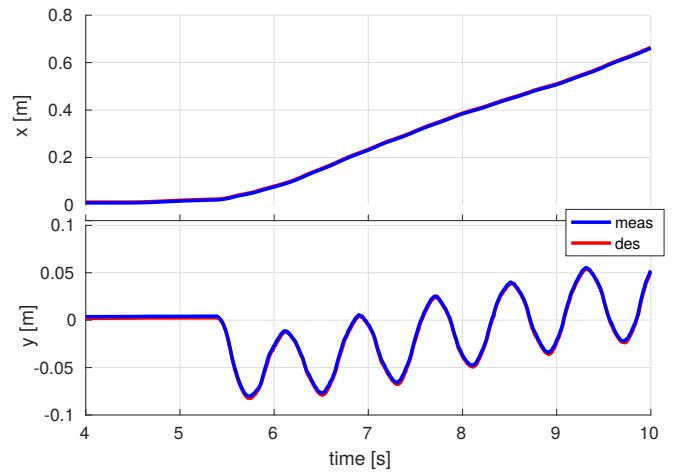


Fig. 5. The time evolution of the measured (blue) and reference (red) whole-body center of mass position on the x and y directions during a pacing gait.

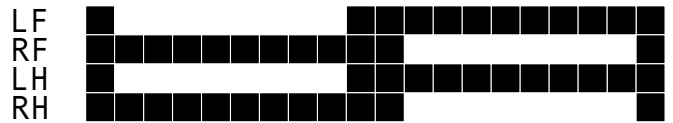


Fig. 6. The contact schedule associated to a pacing gait. This gait exhibits a complete overlap in the swing phase of two legs which are on the same side of the main body, namely fore/hind left or fore/hind right legs.

tested an online transition from stand to a dynamic lateral walk to a pace. This was achieved by applying a transition from the walking to the pacing gait by interpolating the two footfall patterns as a function of the commanded heading speed. As shown in the videos, we can smoothly switch from stand, to dynamic walk to pace and safely back to stand.

VI. CONCLUSIONS AND FUTURE WORK

We have shown how it is possible to execute highly dynamic and agile gaits on a torque-controllable quadruped robot by combining a motion planner based on a simplified dynamic model and a hierarchical whole-body controller. The motion planner based on the ZMP stability criterion is continuously updating the reference trajectory as a function of the current gait pattern and the robot state; the whole-body controller tracks the reference plan by solving a hierarchy of tasks. We have demonstrated the execution of dynamic gaits such as a trot, a dynamic lateral walk, a pace and a transition between the last two on ANYmal. To the best of our knowledge, this is the first time that experimental results are published for such gaits.

The next steps will be focused on improving the main features of gait execution and gait transition. This includes foothold evaluation and gait pattern adaptation in order to be able to deal with harsh changes in the state of the robot and sudden changes in the active gait pattern. The framework should also be extended to include full flight phases, which would allow to perform even more agile locomotion.

¹<https://youtu.be/iUQE-ZQdJY>

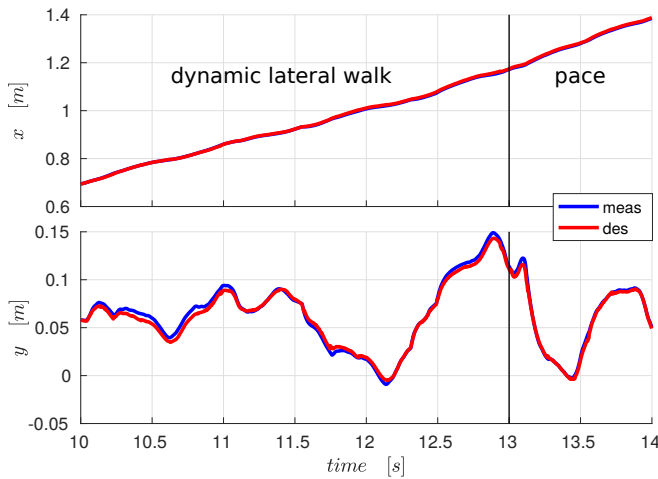


Fig. 7. The time evolution of the measured and reference whole-body center of mass position in the inertial frame during a transition from dynamic lateral walk to pace.

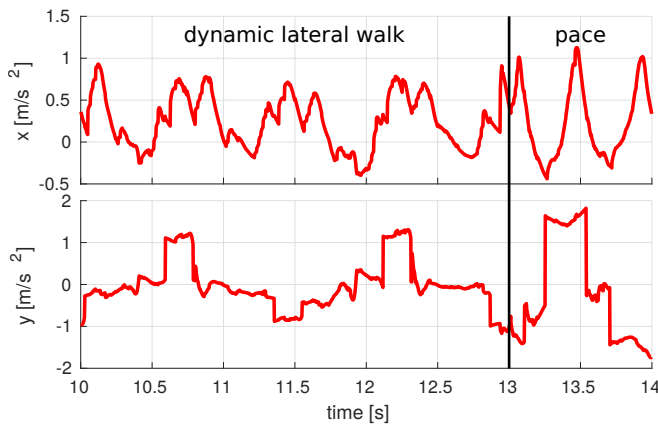


Fig. 8. The time evolution of the reference acceleration for the whole-body center of mass during a transition from dynamic lateral walk to pace.

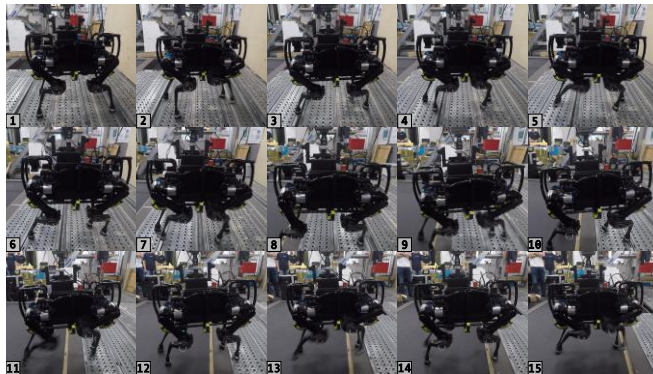


Fig. 9. A sequence showing ANYmal during a transition from a dynamic lateral walk to a pacing gait. Balance is retained even though there is an unperceived step.

REFERENCES

- [1] C. D. Bellicoso, C. Gehring, J. Hwangbo, P. Fankhauser, and M. Hutter. Perception-less terrain adaptation through whole body control and hierarchical optimization. In *2016 IEEE-RAS 16th Int. Conf. on Humanoid Robots (Humanoids)*, pages 558–564, Nov 2016.
- [2] M. Bloesch, C. Gehring, P. Fankhauser, M. Hutter, M. A. Hoepflinger, and R. Siegwart. State estimation for legged robots on unstable and slippery terrain. In *2013 IEEE/RSJ Int. Conf. on Intelligent Robots and Systems*, pages 6058–6064. IEEE, nov 2013.
- [3] P. Fankhauser, C. D. Bellicoso, C. Gehring, R. Dub, A. Gavel, and M. Hutter. Free gait - an architecture for the versatile control of legged robots. In *2016 IEEE-RAS 16th Int. Conf. on Humanoid Robots (Humanoids)*, pages 1052–1058, Nov 2016.
- [4] R. Featherstone. *Rigid Body Dynamics Algorithms*. Springer US, Boston, MA, 2008.
- [5] C. Gehring, S. Coros, M. Hutter, C. D. Bellicoso, H. Heijnen, R. Diethelm, M. Bloesch, P. Fankhauser, J. Hwangbo, M. Hoepflinger, and R. Siegwart. Practice makes perfect: An optimization-based approach to controlling agile motions for a quadruped robot. *IEEE Robotics Automation Magazine*, 23(1):34–43, March 2016.
- [6] D. Goldfarb and a. Idnani. A numerically stable dual method for solving strictly convex quadratic programs. *Mathematical Programming*, 27(1):1–33, 1983.
- [7] S. Gottschalk. Separating axis theorem. Technical report, TR96-024, Department of Computer Science, UNC Chapel Hill, 1996.
- [8] A. Herzog, L. Righetti, F. Grimmering, P. Pastor, and S. Schaal. Balancing experiments on a torque-controlled humanoid with hierarchical inverse dynamics. In *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 981–988, Sept 2014.
- [9] M. Hutter, C. Gehring, D. Jud, A. Lauber, C. D. Bellicoso, V. Tsounis, J. Hwangbo, P. Fankhauser, M. Bloesch, R. Diethelm, and S. Bachmann. ANYmal - A Highly Mobile and Dynamic Quadrupedal Robot. In *IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*, 2016.
- [10] M. Kalakrishnan, J. Buchli, P. Pastor, M. Mistry, and S. Schaal. Fast, robust quadruped locomotion over challenging terrain. In *2010 IEEE Int. Conf. on Robotics and Automation*, pages 2665–2670. IEEE, may 2010.
- [11] M. Kalakrishnan, J. Buchli, P. Pastor, M. Mistry, and S. Schaal. Learning, planning, and control for quadruped locomotion over challenging terrain. *The International Journal of Robotics Research*, 30(2):236–258, nov 2010.
- [12] Luca Di Gaspero. QuadProg++. Available at <http://quadprog.sourceforge.net/>, 1998.
- [13] Martin Felis. Rigid Body Dynamics Library. Available at <http://rbdlib.bitbucket.org/>.
- [14] R. McGhee and A. Frank. On the stability properties of quadruped creeping gaits. *Mathematical Biosciences*, 3:331–351, aug 1968.
- [15] H.-W. Park, P. M. Wensing, and S. Kim. Online planning for autonomous running jumps over obstacles in high-speed quadrupeds. *Robotics: Science and Systems*, 2015.
- [16] P. Sardain and G. Bessonnet. Forces acting on a biped robot. center of pressure-zero moment point. *IEEE Transactions on Systems, Man, and Cybernetics - Part A: Systems and Humans*, 34(5):630–637, Sept 2004.
- [17] C. Semini, N. G. Tsagarakis, E. Guglielmino, M. Focchi, F. Cannella, and D. G. Caldwell. Design of hyq—a hydraulically and electrically actuated quadruped robot. *Proceedings of the Institution of Mechanical Engineers, Part I: Journal of Systems and Control Engineering*, 225(6):831–849, 2011.
- [18] M. Vukobratovic and B. Borovac. Zero-moment point thirty five years of its life. *International Journal of Humanoid Robotics*, 01(01):157–173, 2004.
- [19] A. W. Winkler, C. Mastalli, M. Focchi, D. G. Caldwell, and I. Havoutis. Planning and Execution of Dynamic Whole-Body Locomotion for a Hydraulic Quadruped on Challenging Terrain. *Icra*, pages 5148–5154, 2015.